

# ОС і СП

## Лабораторна робота 12

### Інтерпретація за формулами; операторне виконання

**Мета роботи.** 1) Вивчити теоретичні основи і приклад інтерпретатора формул за алгебраїчними правилами. Доповнити інтерпретатор іншими бінарними і унарними операціями. 2) Реалізувати операторне виконання для операторів присвоєння і `print()` мінімізованого формату мови Python.

#### Матеріали до завдання

- текст лекції "L11 Обчислення формул інтерпретацією.pdf"; в тексті лекції є посилання на файли `extarithexpr.py` і `extarithexpr-ass.py`, ці файли *не надаємо*, вони мали би бути частиною самостійного виконання завдання;
- для поглибленого вивчення питань цієї лабораторної роботи можна звернутись до лекцій "L12 Сканер для формул електронної таблиці.pdf" і "L13 Перетворення виразів ЕТ в постфіксну форму.pdf", в яких подано реалізацію обчислень в форматі електронних таблиць, реалізованих мовою C++; зауважимо, що питання лекцій L12 і L13 включені до екзаменаційних білетів;
- покликання на електронні джерела попередньої лабораторної роботи:  
<https://uk.wikipedia.org/wiki/Інтерпретатор>  
<http://kytok.org.ua/?p=732>  
<https://uk.wikipedia.org/wiki/Компілятор>
- файл `arithexpr.py` прикладу інтерпретатора.

За основу роботи прийняти доданий файл `arithexpr.py` і уважно вивчити алгоритм і особливості реалізації сканування (перший перегляд) і особливості нисхідного граматичного розбору відповідно до граматичних правил визначення формули (другий перегляд). Тут застосовано метод рекурсивного спуску за синтаксичним деревом, який є простіший для реалізації. Деякі важливі особливості коротко описані тут далі в самому завданні, а всі інші - як коментарі в програмному коді файлу `arithexpr.py`.

Граматичні (синтаксичні) правила для формул записані не в класичній формі зображення синтаксису, а різновидом нотації Бекуса-Наура з використанням регулярних виразів:

[https://uk.wikipedia.org/wiki/Регулярний\\_вираз](https://uk.wikipedia.org/wiki/Регулярний_вираз)

За темою граматичного розбору можна зокрема переглянути покликання:

[https://uk.wikibooks.org/wiki/Реалізація\\_алгоритму\\_рекурсивного\\_спуску\\_з\\_прикладями\\_на\\_C%2B%2B](https://uk.wikibooks.org/wiki/Реалізація_алгоритму_рекурсивного_спуску_з_прикладями_на_C%2B%2B)

## Завдання частина 1: вивчення, означення, експерименти

1. В текстовому файлі чи в текстовому рядку записана формула, яка складається з цілих чисел, знаків операцій +, -, \*, /, круглих дужок. Наприклад:

$$49 - 108 / (6 + 11) * (100 - 94)$$

Необхідно виконати обчислення формули за правилами алгебри:

```
arith_expr ::= term ( ( "+" | "-" ) term ) *  
term ::= factor ( ( "*" | "/" ) factor ) *  
factor ::= number | "(" arith_expr ")"  
number ::= cipher cipher *  
cipher ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Аксіомою цієї граматики є `arith_expr`. Реалізацію граматики виразів і розпізнавання виразу реалізуємо нисхідним граматичним розбором за методом *рекурсивного спуску*. За методом рекурсивного спуску до кожного нетермінального символу будують окрему функцію, яка повинна розпізнати праву частину граматичного правила. При цьому функція може викликати в потрібні моменти інші функції для розпізнавання частини конструкції.

Для вивчення методу рекурсивного спуску є такі публікації:

Рекурсивний аналізатор спуску:

<https://www.geeksforgeeks.org/recursive-descent-parser/>

Що таке аналізатор рекурсивного спуску:

<https://www.tutorialspoint.com/what-is-recursive-descent-parser>

Типи аналізаторів у компіляторі:

<https://www.geeksforgeeks.org/types-of-parsers-in-compiler-design/>

Вхідними даними для програми обчислення виразу є масив лексем, побудований сканером. Арифметичний вираз може мати довільний вигляд в межах записаних вище синтаксичних правил. Тому доцільно підготувати список лексем в формі, зручній для синтаксичного розбору. Кожну лексему можна подати у вигляді пари (код, значення), де код – числове позначення лексеми кожного виду.

2. Програмний код мовою Python інтерпретатора алгебраїчних формул є в доданому файлі `arithexpr.py`. Вивчити програмний код. Можна прийняти його за основу або реалізувати іншою алгоритмічною мовою.

3. Зробимо важливі зауваження щодо алгоритму.

3.1. Як було сказано вище, кожна лексема списку є парою (код, значення), що дозволяє спростити перегляд лексем лише за кодами, а значення – за потреби.

3.2. В кінець списку лексем додаємо в методі `scanner(self)` обмежувач списку лексем (`self.empty, '#'`). Це дозволяє уникнути частих перевірок вичерпання списку лексем в кожному методі. У випадку досягнення обмежувача і спроби продовжити аналіз відбудеться генерування помилки і автоматичне припинення процедури розбору і обчислення.

3.3. Метод `GetNextToken(self)` виконує контрольований перехід до наступної лексеми і генерування помилки у випадку необхідності.

3.4. Методи розбору і обчислення `arithexpr()`, `term()`, `factor()` і `GetNextToken()` мають бути дуже точно синхронізовані щодо пересування списком лексем. Перед викликом кожного з методів `arithexpr()`, `term()`, `factor()` черговою лексемою для аналізу має бути готова найперша лексема відповідної частини арифметичного виразу.

4. Виконати обчислювальні експерименти з інтерпретатором формул. По-перше, можна розкоментувати в методі `calc()` оператори `print()`, щоб побачити кроки роботи. Наприклад, для показаної вище формули

$49 - 108 / (6 + 11) * (100 - 94)$

отримаємо такі результати:

`49-108/(6+11)*(100-94)`

```
[ (1, 49), (6, '-'), (1, 108), (8, '/'), (3, '('), (1, 6), (5, '+'),  
  (1, 11), (4, ')'), (7, '*'), (3, '('), (1, 100), (6, '-'), (1, 94),  
  (4, ')'), (0, '#') ]
```

`10.882352941176471`

По-друге, варто записати формулу з синтаксичними помилками і переглянути повідомлення інтерпретатора для випадків помилок.

По-третє, можна додати для експериментів інші оператори `print()` до тексту методів, і отримати спостереження за ходом виконання інтерпретатора.

## Завдання частина 2: розширення граматики і програмування

5. Розширення змісту формул для підготовки задачі. Записані вище правила будови формул доповнити такими можливостями (рекомендуємо в порядку перелічення).

5.1. Додати бінарні операції ділення на ціло `//` і остачу від ділення `%`.

5.2. Додати операцію піднесення до степеня `**`; звернемо увагу, що піднесення до степеня має вищий ранг, ніж операції множення і ділення.

5.3. Додати функцію `sin(x)` і ще одну іншу функцію.

5.4. Додати *унарні* операції `+` і `-`; їх можна записати перед будь-яким співмножником чи доданком; мають вищий ранг від бінарних операцій `*`, `/`, `+`, `-`, але нижчий від операції піднесення до степеня `**`; наприклад `-2+-6`, `4*+8`; отже, якщо записані два знаки операцій підряд, тоді перший знак означає бінарну операцію, а другий – унарну.

5.5. Крім цілих чисел дозволити дійсні числа форматів фіксованої крапки і в експоненціальній формі, наприклад `45.02`, `1.0e-5`, `28e4`; зафіксуємо вимогу, що запис будь-якого числа завжди починається з цифри.

```
arith_expr ::= term ( ( "+" | "-" ) term ) *  
  # term ::= factor ( ( "*" | "/" ) factor ) *      # попереднє  
term ::= factor ( ( "*" | "/" | "//" | "%" ) factor ) * # нове  
  # factor ::= number | "(" arith_expr ")"          # попереднє
```

```

factor ::= [( + | - )] ( number | "(" arith_expr ")" | function )
        [( "***" factor )*]      # нове
number ::= cipher cipher * [ . cipher cipher * ]
        [ (e|E) [(+|-)] cipher cipher * ]
function ::= "sin(" arith_expr ")"
cipher ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

```

Отже, правила змінили так, щоб зберегти раніше визначену частину правил, додати нові операції і врахувати пріоритети нових операцій.

Порівняти новий варіант правил з попереднім. Зробити пропозиції до подальшого розширення формул.

**6. Доопрацювати інтерпретатор arithexpr.py для реалізації перерахованих розширень.**

Перелік можливих змін є такий:

- 1) доповнити сканер новими лексемами;
- 2) додати до методу `term()` нові бінарні операції;
- 3) змінивши правило для `factor`, тепер треба відповідно переписати метод `factor()`. Для цього можна повторювати в циклі (або рекурсивно) основну частину записаної раніше функції і в цій же функції обчислювати піднесення до степеня;
- 4) треба інакше визначити метод `onenumber()`, щоб він дозволяв форму числа цілого і дійсного;
- 5) варто визначити окремий метод `funcname()` для сканера, щоб виділити ім'я функції `sin` чи іншої;
- 6) до методу `factor()` додати можливість унарної операції `+` або `-`.

### **Завдання частина 3: операторне виконання**

**7. Основні міркування щодо цієї частини роботи викладені в розділі "Операторне виконання" лекції L11. Там же подано визначення граматики. Вивчити граматiku і розуміти, як відбувається виведення (підстановка правил), починаючи від аксіоми `prog`.**

Питання будови алгоритмічних мов, граматик, виведення, розпізнавання, проектування компіляторів тощо виходять за межі нашого курсу. Тому подаємо покликання для самостійного ознайомлення з подібними проблемами:

Вступ до синтаксичного аналізу в проектуванні компілятора:

<https://www.geeksforgeeks.org/introduction-to-syntax-analysis-in-compiler-design/>

Проектування компілятора - синтаксичний аналіз:

[https://www.tutorialspoint.com/compiler\\_design/compiler\\_design\\_syntax\\_analysis.htm](https://www.tutorialspoint.com/compiler_design/compiler_design_syntax_analysis.htm)

Лексичний аналіз і синтаксичний аналіз:

<https://www.geeksforgeeks.org/lexical-analysis-and-syntax-analysis/>

Вікіпедія. Розбір:

<https://en.wikipedia.org/wiki/Parsing>

MREC Academics Login. Module II: Syntax Analysis:

<https://mrecacademics.com/DepartmentStudyMaterials/20230414151537-Compiler%20Design%20Module%20%20Notes.pdf>

8. Реалізувати свій варіант інтерпретатора для операторного виконання, враховуючи результати попередніх пунктів завдання. Підготувати для тестування 2 варіанти прикладів програми (вхідних даних), подібно як в згаданому розділі.

Підказка: результат інтерпретації власного виконання має співпасти з результатом виконання стандартного інтерпретатора системи програмування, якщо тестові варіанти прикладів записати в окремий файл і виконати в системі, так, як ми працюємо в звичайному режимі розробки програм.

---

#### **Завдання частина 4 – факультативно, воно ж є завданням на проєкт за цілим розділом**

Доповнити граматику і реалізувати програмно інтерпретатор ~Python-мови на основі попередніх виконаних завдань за такими пунктами:

- всі обчислення реалізувати в межах числових типів і логічних;
- додати оператор `input()` в мінімальній формі, без підказок на екран, прочитаний текстовий рядок зразу переводити до числової форми або додатково реалізувати функції `int()` та `float()`;
- додати означення блоку – послідовності операторів з відступами тексту ліворуч;
- визначити поняття "логічний вираз" окремо від арифметичного, це може бути корисним для операторів `if` та `while`;
- додати умовний оператор `if-else` без розширень;
- додати оператор циклу `while`.

Реалізація операторів `if`, `while` є комбінацією логічної перевірки умови, присвоєнь і безумовного переходу `goto` - подібно, як малюють блок-схему виконання такого оператора.

За бажанням можна додати інші елементи і отримати значний досвід проєктування і програмування подібних задач.

Можна просто розширити самотійно граматики, подані в лекції L11. Або звернутись до повного граматичного визначення мови Python за документацією, відібрати і скоротити необхідні елементи:

<https://docs.python.org/3/reference/grammar.html>

У випадку реалізації необхідно подати такі результати за частиною 4:

- тестовий файл свого остаточного варіанта визначення граматики;
- повний програмний код інтерпретатора з обов'язковим коментуванням, або архів проєкту, якщо реалізація виконана іншою мовою;
- три різні тестові приклади ~python-програм, які б включали в сукупності застосування всіх елементів своєї граматики;
- отримані результати інтерпретації.

Насамкінець зауважимо, що значна частина ідей, методів і підходів, викладених в лабораторній роботі 12, реалізується в архітектурах і технологіях макросів чи скриптів – що стосується безпосередньо системного програмування.

В широкому спектрі задачі про застосування синтаксичного аналізу можна переглянути за покликанням:

<https://www.coursera.org/articles/syntax-analysis>

---

### Звіт за роботу (частини 1, 2, 3)

В результаті виконання роботи надіслати:

1) програмний код файлу python доопрацьованого інтерпретатора або архів проєкту, якщо реалізація виконана іншою мовою; це має бути програмна реалізація п.5 завдання - розширення змісту формул, можливий варіант якої описаний в п.6; звертаємо увагу, що програмний код треба обов'язково коментувати за змістом доопрацювання;

2) тестові приклади формул, на яких виконали перевірку інтерпретатора за пунктом 6; перелік тестів має достатнім для повної демонстрації всіх елементів завдання за пунктом 6;

3) роздрук отриманих результатів разом з коментарями – протокол виконання на своєму комп'ютері за пунктом 6; нагадуємо, що коментарі є обов'язковим елементом подання результатів в прикладному і системному програмуванні;

4) аналогічно – програмний код інтерпретатора за пунктом 8 завдання;

5) тестові приклади зразків програм і отримані результати інтерпретації за пунктом 8 завдання.

|                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Увага!</b> Окремі файли не архівувати, за винятком, можливо, архіву цілого проєкту у випадках, коли процедура розробки передбачає багатофайлові проєкти. Тестові приклади формул і файли отриманих результатів незалежно надіслати окремими вкладеннями для попереднього перегляду змісту виконаної роботи.</p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|